New threads are created with the Thread.new call. The Thread.new call is given a block that contains the code to be run in the new thread. In our case, the block uses the net/http library to fetch the page from the URL that is passed to the thread. Once the thread is created, it is available to be scheduled for execution by the operating system, and, in this code at least, we do not have direct control over when the thread runs.

This code uses a map call to create three new threads from the list of sites to call and stores the threads in an array. Threads, like everything else in Ruby, are objects that can be assigned to variables, returned from blocks or methods, and passed as parameters.

Our output tracing shows that these fetches are going on in parallel, because all three "fetch" statements happen before any of the "got" statements do. Broadly, what's happening is that the first thread is being created, is scheduled for control, makes its HTTP request, is blocked while it waits for the answer, and control reverts back to the main program, which immediately creates the second thread, and so on. The thread creation is much faster than the HTTP request, so even with the overhead of making the threads, all three threads will usually be created before any of them return. Just to be clear, we are dealing with one word and two meanings: "block" means both "a chunk of Ruby code" and "being stalled waiting for a response."

When we create the thread, we pass the required URL as a parameter to the block as url, even though the same value is already available as page_to_fetch outside the block. Why do we do this? The answer relates to thread safety and how threads share values.

A thread shares all global, instance, and local variables that are in existence and available at the time the thread starts. Despite what Mr. Rogers says, sharing isn't always a good thing. In this case, all three threads share the variable page_to_fetch, defined outside the Thread.new block. The first thread gets started, and page_to_fetch is set to "www.rubycentral.org". In the meantime, the loop creating the threads is still running. The second time around, page_to_fetch gets set to "pragprog.com". If the first thread has not yet finished using the page_to_fetch variable, it will suddenly start using this new value. In our case, that would likely manifest as using one value in the actual http.get command, and then the value changing while the thread was blocked, and a different value being used in the print statement on the next line. This would be, to say the least, confusing. These kinds of bugs are difficult to track down.

However, local variables created within a thread's block are truly local to that thread—each thread will have its own copy of these variables. In our case, the variable url will be set at the time the thread is created, and each thread will have its own copy of the page address. You can pass any number of arguments into the block via Thread.new, the arguments to the method become the arguments to the block.

This code also illustrates a gotcha. Inside the loop, the threads use print to write out the messages, rather than puts. Why? Well, behind the scenes, puts splits its work into two chunks: it writes its argument, and then it writes a newline. Between these two, a thread could get scheduled, and the output would be interleaved. Calling print with a single string that already contains the newline gets around the problem.

Manipulating Threads

Another subtlety occurs in the next-to-last line of our program, where we call join on each thread.